

Absolutely. Since you're learning Salesforce/Apex practically, the easiest way is to understand **why asynchronous Apex exists first**, and then learn the four mechanisms one by one.



Asynchronous Apex — Complete Beginner-Friendly Guide

The four things you need to learn are:

1. **Future Method**
2. **Queueable Apex**
3. **Batch Apex**
4. **Schedulable Apex**

Think of them as **four different ways of telling Salesforce:**

"Don't make the user wait while you do this work. Do it in the background."

1. First: What is Asynchronous Apex?

Normally, Apex runs **synchronously**.

Synchronous

Imagine you're at a restaurant.

You order:

"Give me a pizza."

The waiter says:

"Sure, wait here."

You stand there until the pizza is prepared.

That's synchronous execution.

In Salesforce:

```
User clicks Save
      ↓
Apex executes
      ↓
Apex finishes
```

The user has to wait for the transaction to finish.

Asynchronous

Now imagine:

You order the pizza.

The waiter says:

"Your order is placed. You can go. We'll prepare it in the background."

That's asynchronous.

```
User clicks Save
      ↓
Apex starts background job
      ↓
User gets response
      ↓
Salesforce processes job later
```

The important idea:

Asynchronous Apex allows Salesforce to perform work later/in the background rather than making the current user transaction wait for it.

2. Why do we need Asynchronous Apex?

There are several common reasons.

Example 1 — Huge amount of data

Suppose your company has:

1 million Accounts

and you want to update all of them.

Doing this in one normal transaction isn't practical.

You could use:

Batch Apex

to process them in smaller groups.

1,000,000 Accounts

↓

Batch

↓

200 records

200 records

200 records

...

Example 2 — External API

Suppose Salesforce needs to send information to an external payment system.

You don't want the user to sit there waiting for an external API response.

You can use asynchronous processing.

Example 3 — Complex processing

Suppose a user creates an Application.

Salesforce needs to:

- calculate something
- update several records
- send information externally
- perform additional processing

Instead of making the user wait, some of that work can happen asynchronously.

Example 4 — Run something at a particular time

Suppose:

"Every day at 2 AM, update expired job postings."

You don't want a human to click anything.

Use:

Schedulable Apex

The four types in one sentence

Memorize this:

Type	Think
Future	"Do this later."
Queueable	"Put this job in a queue."
Batch	"Process a LOT of records in chunks."
Schedulable	"Run this at a specific time."

Now let's understand each one properly.

3. FUTURE METHOD

What is a Future Method?

A Future Method is a method that runs **asynchronously in the future**.

Instead of executing immediately:

```
Method called
  ↓
Execute now
```

you essentially tell Salesforce:

```
Method called
  ↓
Put it in background
  ↓
Execute later
```

Real-life example

Imagine you're working at a recruiting company.

A candidate applies for a job.

When they apply, Salesforce needs to send their information to an external recruitment system.

Instead of making the candidate wait:

```
Candidate applies
    ↓
Application saved
    ↓
User gets response
    ↓
Future method runs
    ↓
External system updated
```

Basic Future syntax

apex

```
public class CandidateService {

    @future
    public static void sendCandidate(Id candidateId) {

        System.debug('Processing candidate: ' + candidateId);

    }
}
```

You call it:

apex

```
CandidateService.sendCandidate(candidateId);
```

But because of:

apex

```
@future
```

Salesforce runs it asynchronously.

Important Future rules

This is very important for interviews.

1. Future method must be static

apex

```
@future
public static void myMethod() {
}
```

Not:

apex

```
@future
public void myMethod() {
}
```

2. It returns void

apex

```
@future
public static void myMethod() {
}
```

You can't do:

apex

```
@future
public static String myMethod() {
}
```

3. Parameters must be primitive types

For example:

apex

```
@future
public static void processAccount(Id accountId) {
```

```
}
```

You can use:

- String
- Integer
- Boolean
- Decimal
- Id
- etc.

But you shouldn't pass an entire Account object.

Instead of:

apex

```
processAccount (account) ;
```

pass:

apex

```
processAccount (account.Id) ;
```

Then query the record inside the future method.

Why?

Because the actual execution happens later.

The Account could potentially change between the time you call the method and the time Salesforce executes it.

So you usually pass the **Id** and query fresh data.

Future method: when should I use it?

Good use cases:

External callout

Salesforce

↓

Future method

Simple background processing

For example:

When an Account is created, perform some additional processing later.

Future limitations

Future methods are relatively old compared with Queueable Apex.

They have limitations such as:

- no job chaining in the way Queueable supports
- limited parameter types
- no easy job monitoring compared with Queueable
- harder to structure complex asynchronous processes

So today, if you need a more sophisticated asynchronous job, **Queueable is often preferable**.

4. QUEUEABLE APEX

Now we come to one of the most important ones.

What is Queueable Apex?

Think of a **queue at an airport**.

People arrive:

Person A
Person B
Person C
Person D

They wait in line.

Salesforce can similarly place jobs into a queue.

Queue

Job A
Job B
Job C
Job D

Salesforce processes them asynchronously.

Real-life Salesforce example

Suppose:

A candidate gets hired.

Now Salesforce needs to:

1. Update candidate status
2. Update job position
3. Send information to HR system
4. Create employee record

That's a reasonably complex job.

You can create a Queueable class.

Basic Queueable code

```
apex

public class CandidateQueue implements Queueable {

    private Id candidateId;

    public CandidateQueue(Id candidateId) {
        this.candidateId = candidateId;
    }

    public void execute(QueueableContext context) {

        System.debug('Processing candidate: ' + candidateId);

    }
}
```

Then enqueue it:

```
apex
```

```
System.enqueueJob(  
    new CandidateQueue(candidateId)  
);
```

Understand this line

apex

```
System.enqueueJob(  
    new CandidateQueue(candidateId)  
);
```

You're basically saying:

"Salesforce, please put this job in the asynchronous queue."

Salesforce then executes:

apex

```
execute()
```

later.

Why Queueable is powerful

Queueable supports **objects as member variables** and allows more structured jobs than Future methods.

For example:

apex

```
public class CandidateQueue implements Queueable {  
  
    private Id candidateId;  
    private String status;  
  
    public CandidateQueue(Id candidateId, String status) {  
        this.candidateId = candidateId;  
        this.status = status;  
    }  
  
    public void execute(QueueableContext context) {
```

```
        System.debug(candidateId);  
        System.debug(status);  
  
    }  
}
```

Queueable chaining

This is a VERY important concept.

Suppose:

```
Job 1  
↓  
Job 2  
↓  
Job 3
```

You can have one Queueable job enqueue another Queueable job.

Example:

```
apex  
  
public class JobOne implements Queueable {  
  
    public void execute(QueueableContext context) {  
  
        System.debug('Job 1');  
  
        System.enqueueJob(new JobTwo());  
    }  
}
```

So:

```
Queueable 1  
↓  
Queueable 2  
↓  
Queueable 3
```

This is called **Queueable chaining**.

Queueable vs Future

This is one of the most important interview questions.

Future	Queueable
Older/simple mechanism	More modern/flexible
<code>@future</code>	implements <code>Queueable</code>
Static method	Class + <code>execute()</code>
Primitive parameters	Can maintain richer state
Limited flexibility	More flexible
Harder to chain	Supports chaining
Basic async work	Complex async jobs

Easy memory trick

Future = simple background task

Queueable = structured background job

5. BATCH APEX

Now let's understand the big one.

What is Batch Apex?

Batch Apex is used when you need to process **large amounts of data**.

This is its primary purpose.

Imagine your company has:

5 million Accounts

You can't realistically process all of them in one transaction.

Instead:

5,000,000 records

↓

Batch Apex

↓

200 records

200 records

200 records

200 records

...

Each chunk is processed separately.

Real-life example

Suppose your company has **500,000 job applications**.

You want to change:

```
Status = "Old"
```

for applications older than one year.

Batch Apex is perfect.

Basic Batch structure

A Batch class typically implements:

```
apex
```

```
Database.Batchable<SObject>
```

It has three important methods:

```
start()  
execute()  
finish()
```

These three are extremely important.

1. start()

`start()` determines **which records should be processed**.

Example:

apex

```
global Database.QueryLocator start(  
    Database.BatchableContext bc  
) {  
  
    return Database.getQueryLocator(  
        'SELECT Id, Status__c FROM Job_Application__c'  
    );  
}
```

Think:

"Which records do I want?"

2. execute()

`execute()` processes each chunk.

Example:

apex

```
global void execute(  
    Database.BatchableContext bc,  
    List<Job_Application__c> applications  
) {  
  
    for (Job_Application__c app : applications) {  
        app.Status__c = 'Old';  
    }  
  
    update applications;  
}
```

Think:

"What should I do with these records?"

3. finish()

Runs after the entire batch has completed.

apex

```
global void finish(Database.BatchableContext bc) {  
  
    System.debug('Batch completed');  
}
```

Think:

"What should I do after everything is finished?"

Complete Batch Example

apex

```
global class UpdateApplications  
    implements Database.Batchable<SObject> {  
  
    global Database.QueryLocator start(  
        Database.BatchableContext bc  
    ) {  
  
        return Database.getQueryLocator(  
            'SELECT Id, Status__c FROM Job_Application__c'  
        );  
    }  
  
    global void execute(  
        Database.BatchableContext bc,  
        List<Job_Application__c> applications  
    ) {  
  
        for (Job_Application__c app : applications) {  
            app.Status__c = 'Old';  
        }  
  
        update applications;  
    }  
  
    global void finish(Database.BatchableContext bc) {  
  
        System.debug('Batch completed');  
    }  
}
```

Run it:

apex

```
Database.executeBatch(  
    new UpdateApplications(),  
    200  
);
```

The `200` is the **batch size**.

Meaning:

Process up to 200 records per execution.

Visualizing Batch Apex

Suppose you have:

1,000 records

and batch size is:

apex

200

Salesforce processes:

start()

↓

200 records → execute()

↓

200 records → execute()

↓

200 records → execute()

↓

200 records → execute()

↓

200 records → execute()

↓

That's the core concept.

When should you use Batch Apex?

Whenever the main problem is:

"I have a large number of records to process."

Examples:

Example 1

Update 2 million Accounts.

→ Batch

Example 2

Delete old records.

→ Batch

Example 3

Recalculate information for hundreds of thousands of records.

→ Batch

Example 4

Send/process data for a huge number of records.

→ Batch

6. SCHEDULABLE APEX

Now something different.

Schedulable Apex isn't primarily about **how many records** you're processing.

It's about:

WHEN should the code run?

Real-life example

Suppose your HR system needs to run:

Every day at 2 AM.

It checks expired job postings.

You don't want an employee to manually run it.

You schedule it.

Basic Schedulable Apex

apex

```
global class ExpiredJobsScheduler
    implements Schedulable {

    global void execute(SchedulableContext sc) {

        System.debug('Running scheduled job');

    }
}
```

Then schedule it:

apex

```
String cronExpression = '0 0 2 * * ?';

System.schedule(
    'Expired Jobs Daily',
    cronExpression,
    new ExpiredJobsScheduler()
);
```

This tells Salesforce:

Run this at 2 AM every day.

Think about Schedulable like an alarm clock

Your phone alarm:

Set alarm

↓

2:00 AM

↓

Alarm rings

Schedulable Apex:

Set schedule

↓

2:00 AM

↓

Apex runs

Schedulable + Batch

This is VERY common.

You can schedule a Batch job.

For example:

Every night at 2 AM

↓

Schedulable Apex

↓

Starts Batch Apex

↓

Processes 500,000 records

This is a real-world Salesforce architecture.

Example:

apex

```
global class NightlyJob implements Schedulable {
```

```
global void execute(SchedulableContext sc) {  
  
    Database.executeBatch(  
        new UpdateApplications(),  
        200  
    );  
}  
}
```

Then schedule it.

★ The BIG Difference

Now let's put all four together.

Type	Main purpose	Simple meaning
Future	Simple async work	Do it later
Queueable	Complex async work	Put this job in a queue
Batch	Large data	Process lots of records in chunks
Schedulable	Timing	Run this at a specific time

🔥 Real-life scenario comparison

Imagine you run a recruitment company.

You have:

Candidates

Job Applications

Positions

Now look at these situations.

Scenario 1

A candidate applies.

You need to call an external API to notify another system.

Use:

Future

```
Candidate applies
      ↓
Application saved
      ↓
Future method
      ↓
External API
```

Simple asynchronous task.

Scenario 2

A candidate is hired.

You need to:

- update application
- update position
- create employee information
- call external system
- perform several processing steps

Use:

Queueable

```
Candidate hired
      ↓
Queueable job
      ↓
Process
      ↓
Next Queueable if needed
```

Scenario 3

Your company has:

2 million old applications

and you need to archive them.

Use:

Batch

```
2 million records
    ↓
Batch
    ↓
200
200
200
200
...
```

Scenario 4

Every Sunday at midnight, you want to clean old records.

Use:

Schedulable

```
Sunday midnight
    ↓
Scheduler
    ↓
Run cleanup
```

Scenario 5 — Combination

Now suppose:

Every night at 2 AM, process 1 million old applications.

You could use:

```
Schedulable
    ↓
Batch
    ↓
Process records in chunks
```

This is a very realistic Salesforce architecture.



The easiest way to decide

When you get a question in an interview or coding challenge, ask yourself:

Question 1:

Do I need to run this in the background?

If no:

→ Normal synchronous Apex.

If yes:

→ Async Apex.

Question 2:

Do I have a huge amount of records?

Yes:

→ **Batch Apex**

Question 3:

Does it need to run at a particular time?

Yes:

→ **Schedulable Apex**

Question 4:

Is it a relatively simple background operation?

Yes:

→ **Future Method**

Question 5:

Is it a more complex background operation that may need chaining/state?

Yes:

→ **Queueable Apex**

Interview Cheat Sheet

Memorize this table:

Question	Answer
Simple async operation?	Future
Complex async operation?	Queueable
Huge amount of data?	Batch
Run at specific time?	Schedulable
Need chaining?	Queueable
Need chunk-based processing?	Batch
Need <code>start/execute/finish</code> ?	Batch
Need <code>execute()</code> with <code>SchedulableContext</code> ?	Schedulable
Uses <code>@future</code> ?	Future
Uses <code>System.enqueueJob()</code> ?	Queueable
Uses <code>Database.executeBatch()</code> ?	Batch
Uses <code>System.schedule()</code> ?	Schedulable

One final mental model

Imagine **four employees**:

 **Future**

"Give me a small task. I'll do it later."

Queueable

"Give me a proper job. I'll put it in my queue and can start another job afterward."

Batch

"Give me the 2 million records. I'll divide them into small groups and process them."

Scheduler

"Tell me WHEN you want something done, and I'll start it at that time."

That's essentially the entire concept.

What you should learn next

For Salesforce interviews and practical coding, I'd learn them in this order:

1. Future Method



2. Queueable Apex



3. Batch Apex



4. Schedulable Apex



5. Future vs Queueable



6. Queueable vs Batch



7. Batch + Scheduler combination



8. Governor limits in asynchronous Apex



9. Test classes for all four



10. Real-world scenario questions

The **test-class part is especially important**, because once you understand these four, you'll start getting questions like *"Write a Queueable class to update X records and write its test class"*—which is very similar to the Apex testing work you've been doing recently.